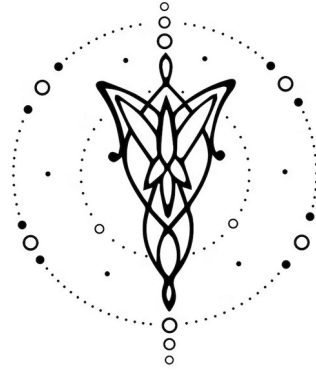




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

3 de Septiembre de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	1
4. Construcciones	2
5. Casos de prueba	4
6. Ejemplos	4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Máximo	Ramos	64.082	maramos@itba.edu.ar
Nicolás	Rodriguez Falcon	63.538	nrodriguezfalcon@itba.edu.ar
Juan Cruz	Procaccini	64.571	jprocaccini@itba.edu.ar

2. Repositorio

La solución será versionada en: https://github.com/MaxiRamoss/TPE_TLA

3. Dominio

Desarrollar un lenguaje específico de dominio que permita describir, componer y simular redes viales urbanas. El lenguaje debe permitir declarar intersecciones y calles dirigidas, programar el comportamiento de los semáforos que las regulan, definir rutas y flujos vehiculares que las recorren, y obtener a partir de todo ello una simulación visual junto con un reporte de métricas de circulación.

La planificación del tránsito exige evaluar intervenciones antes de ejecutarlas: modificar el sentido de una calle, reprogramar los tiempos de un semáforo o cerrar un tramo por obra. Cada una de estas decisiones tiene un costo alto y una reversibilidad baja, y sus efectos no son locales, ya que la congestión se propaga por la red. Actualmente estos escenarios se construyen mediante interfaces gráficas o archivos de configuración

extensos y de bajo nivel, con lo cual no se versionan, no se testean, no se comparan sistemáticamente entre sí y no se pueden generar de forma programática; un cambio menor en la topología obliga a rehacer buena parte de la configuración a mano.

El lenguaje propuesto abstrae este problema permitiendo describir una red vial y su comportamiento dinámico mediante sentencias declarativas de alto nivel, sin que el usuario deba interactuar directamente con el motor de simulación, con su representación interna ni con la lógica de animación y de cálculo de métricas. Un escenario de tránsito pasa así a ser un artefacto de software, con todo lo que ello implica: control de versiones, revisión por pares, pruebas automatizadas, generación paramétrica y reutilización.

Para reducir la complejidad del proyecto, el modelo de simulación se restringe deliberadamente. El tiempo y el espacio son discretos: cada calle es una secuencia de celdas y el tiempo avanza en ticks, de modo que en cada tick un vehículo avanza una celda si la siguiente se encuentra libre y el semáforo que lo habilita está en verde. El modelo es, en esencia, un autómata celular unidimensional, con lo cual no intervienen consideraciones cinemáticas de ningún tipo. Cada calle posee un único carril y no se admite el sobrepaso, de manera que los vehículos conservan su orden relativo; una calle bidireccional se modela como dos calles opuestas. El ruteo es estático, es decir, las rutas se declaran explícitamente o se resuelven una única vez durante la compilación, sin costos asociados a la congestión. La generación de vehículos es determinística, sin distribuciones de probabilidad, con lo cual toda simulación resulta reproducible y, por lo tanto, verificable mediante pruebas automatizadas.

Estas restricciones acotan el alcance del modelo, pero no la expresividad del lenguaje: la sintaxis está diseñada para admitir extensiones como el sobrepaso o el ruteo dinámico sin requerir modificaciones sobre la gramática.

El compilador no ejecuta la simulación. En su salida emite un artefacto HTML autocontenido, con la red representada en SVG y animada mediante JavaScript sobre un runtime provisto por el equipo, junto con un reporte de métricas en formato JSON apto para ser consumido por otras herramientas o para comparar escenarios de manera automatizada. Esta separación mantiene al compilador concentrado en el análisis y la generación de código, y permite evolucionar el modelo de simulación sin alterar el lenguaje.

La implementación satisfactoria de este lenguaje permitiría experimentar diferentes configuraciones de una red vial, verificar sus ventajas, desventajas y puntos de congestión antes de ejecutar la intervención sobre la ciudad, minimizando los riesgos y costos finales.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán declarar una o varias intersecciones, cada una identificada por un nombre único dentro del programa.
- (II). Opcionalmente, se podrá indicar la posición de una intersección mediante `at (x, y)`, con coordenadas enteras expresadas en unidades abstractas y eje vertical creciente hacia abajo. La preposición `at` es locativa y única en el lenguaje: ubica una entidad respecto de otra, ya sea una intersección respecto de un par de coordenadas o un semáforo respecto de una intersección. Las intersecciones que no declaren posición serán ubicadas por el generador de salida mediante un recorrido en anchura determinístico sobre el grafo, de modo que la disposición resultante sea siempre reproducible.
- (III). Toda entidad declarable admitirá opcionalmente una etiqueta descriptiva mediante `label "..."`, empleada exclusivamente para su presentación en el artefacto de salida. De este modo los identificadores permanecen restringidos a caracteres ASCII sin que ello limite la legibilidad del resultado.
- (IV). Se podrán declarar calles dirigidas entre dos intersecciones mediante la construcción `from ... to ...`, indicando su longitud expresada en celdas.
- (V). Las entidades del dominio serán de tipo *intersection*, *road*, *light*, *route*, *flow* o *policy*, y convivirán con los tipos fundamentales *integer*, *boolean* y *string*.

- (VI). Las variables podrán ser arreglos de alguno de los tipos anteriores, y sus elementos serán referenciables mediante un índice.
- (VII). El lenguaje poseerá tipado estático, de modo que la totalidad de las referencias, las incidencias del grafo vial y la compatibilidad de tipos serán validadas durante la compilación, antes de emitir artefacto alguno.
- (VIII). Se proveerá una estructura de control iterativa FOR acotada, resuelta y desenrollada en tiempo de compilación, que permitirá generar corredores y grillas sin enumerar cada elemento individualmente.
- (IX). Se podrán declarar semáforos asociados a una intersección mediante la preposición **at**, compuestos por una secuencia de fases con duración explícita en ticks.
- (X). Se podrán definir *policies*, es decir, procedimientos parametrizados y reutilizables que ajusten el comportamiento de un semáforo, y aplicarlos sobre uno o varios de ellos.
- (XI). Se proveerá una estructura de control condicional de tipo IF-THEN-ELSE dentro de las reglas de semáforo, evaluada una vez por tick.
- (XII). Se proveerán acciones sobre las fases de un semáforo, tales como **extend**, **shorten**, **skip** y **keep**.
- (XIII). Se proveerán funciones de consulta sobre el estado de la simulación, a saber **queue**, **occupancy**, **waiting** y **tick**, utilizables dentro de las reglas de semáforo.
- (XIV). Se podrán declarar rutas como secuencias de intersecciones conectadas entre sí.
- (XV). Se podrán declarar flujos vehiculares sobre una ruta, indicando tasa de emisión mediante **spawn ... every ...** y ventana temporal de actividad mediante **from ... to**
- (XVI). Se proveerán operadores relacionales como **<**, **>**, **=**, **≠**, **≤** y **≥**.
- (XVII). Se proveerán operaciones aritméticas básicas como **+**, **-**, ***** y **/**.
- (XVIII). Se proveerán operaciones lógicas básicas como **AND**, **OR** y **NOT**.
- (XIX). Se proveerá una construcción **log** con interpolación posicional de argumentos, que permitirá trazar el comportamiento de la simulación durante su ejecución.
- (XX). Se proveerá un bloque **report** que seleccionará las métricas a incluir en el artefacto de salida, a saber cantidad de vehículos que completaron su ruta, tiempo promedio de viaje medido en ticks y ocupación máxima por calle.
- (XXI). El lenguaje no proveerá iteración no acotada. La decisión es deliberada: dado que las reglas de semáforo se evalúan una vez por tick, una iteración cuyo límite no pueda determinarse estáticamente permitiría que un único tick no termine. Al restringir la iteración al FOR desenrollado en tiempo de compilación, se garantiza que cada tick consuma tiempo constante y que toda simulación finalice en una cantidad de pasos conocida de antemano.
- (XXII). El lenguaje validará semánticamente que las calles y rutas referencien intersecciones existentes, que los tramos consecutivos de una ruta se encuentren efectivamente conectados, que las fases de un semáforo operen sobre calles que inciden en su intersección, que las longitudes y duraciones sean estrictamente positivas y que no existan identificadores duplicados.
- (XXIII). Se aceptarán comentarios de línea precedidos por **//**, los cuales serán descartados por el analizador léxico y no se verán reflejados en la salida.
- (XXIV). Las palabras reservadas del lenguaje serán en inglés y los identificadores se restringirán a caracteres ASCII, es decir letras, dígitos y guion bajo, comenzando siempre por una letra. De este modo el analizador léxico no queda sujeto a la codificación del archivo fuente. Los literales de cadena, en cambio, admitirán cualquier carácter, dado que se delimitan mediante comillas y se propagan sin interpretar hasta el artefacto de salida.
- (XXV). El compilador traducirá el programa fuente a un artefacto HTML autocontenido, con la red representada en SVG y animada mediante JavaScript, junto con un reporte de métricas en formato JSON.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de aceptación:

- (I). Un programa que declare una única intersección aislada.
- (II). Un programa que declare dos intersecciones unidas por una calle dirigida.
- (III). Un programa que declare una calle indicando su longitud explícita en celdas.
- (IV). Un programa que declare una ruta compuesta por dos tramos consecutivos.
- (V). Un programa que declare un flujo vehicular con tasa de emisión constante.
- (VI). Un programa que declare un semáforo con dos fases de duración fija.
- (VII). Un programa que declare un arreglo de intersecciones y lo recorra mediante FOR para generar un corredor.
- (VIII). Un programa que utilice una expresión aritmética en la duración de una fase.
- (IX). Un programa que defina una *policy* parametrizada y la aplique sobre un semáforo.
- (X). Un programa que utilice IF-THEN-ELSE dentro de una regla de semáforo.
- (XI). Un programa que emita una traza mediante `log` con interpolación de argumentos.
- (XII). Un programa que declare un bloque `report` con las tres métricas disponibles.
- (XIII). Un programa que aplique una misma *policy* sobre dos semáforos con umbrales diferentes.
- (XIV). Un programa que declare una calle bidireccional mediante dos calles de sentidos opuestos.
- (XV). Un programa que combine intersecciones con posición explícita y otras sin declararla.
- (XVI). Un programa que asocie etiquetas descriptivas a intersecciones y calles.
- (XVII). Un programa que combine cualquiera de los casos anteriores dentro de un mismo escenario.

Además, los siguientes casos de prueba de rechazo:

- (I). Un programa malformado.
- (II). Un programa que declare una calle referenciando una intersección inexistente.
- (III). Un programa que declare una ruta cuyos tramos consecutivos no se encuentran conectados.
- (IV). Un programa que declare una fase sobre una calle que no incide en la intersección del semáforo.
- (V). Un programa que declare una calle de longitud cero o negativa.
- (VI). Un programa que declare dos entidades distintas con el mismo identificador.
- (VII). Un programa que invoque una *policy* pasando una intersección donde se espera una calle.
- (VIII). Un programa que declare un flujo cuya ventana temporal finalice antes de comenzar.
- (IX). Un programa que indexe un arreglo fuera de sus límites declarados.
- (X). Un programa que declare dos intersecciones distintas en una misma posición.

Se advierte que, con excepción del primero, los casos de rechazo dependen del análisis semántico y de la validación del grafo vial. En consecuencia, durante el Stage II se aceptarán momentáneamente como falsos positivos, y su rechazo efectivo se implementará en el Stage III.

6. Ejemplos

En el **Cód. (6.1)**, se puede ver la declaración de un escenario mínimo completo: un corredor de tres intersecciones enlazadas por dos calles, un semáforo de dos fases en la intersección central y un flujo vehicular

que recorre el corredor de extremo a extremo. Nótese que cada intersección se nombra por el par de calles que se cruzan en ella, de modo que el propio identificador documenta la topología de la red.

```
// Duración total de la simulación, en ticks:
simulation Centro { ticks 500; }

// Cada intersección se nombra por el cruce que representa.
intersection CordobaPueyrredon at (60, 150);
intersection CordobaCallao      at (430, 150);
intersection CordobaMontevideo at (800, 150);

// Arcos dirigidos; length se expresa en celdas, no en metros.
road CordobaOeste from CordobaPueyrredon to CordobaCallao      length 20;
road CordobaEste  from CordobaCallao      to CordobaMontevideo length 15;

// Semáforo de ciclo fijo: las fases se alternan en orden y
// el ciclo completo dura la suma de sus duraciones (18 ticks).
light at CordobaCallao {
    phase green for CordobaOeste during 10;    // ticks en verde
    phase green for CordobaEste  during 8;
}

// Secuencia de intersecciones conectadas entre sí:
route Corredor = CordobaPueyrredon -> CordobaCallao
                -> CordobaMontevideo;

flow HoraPico along Corredor {
    spawn 2 every 5;    // 2 vehículos cada 5 ticks
    from 0 to 200;      // ventana temporal de actividad
}
```

Código 6.1: Escenario mínimo con un semáforo de ciclo fijo.

En el **Cód. (6.2)**, se emplea un arreglo junto con la construcción iterativa FOR para generar un corredor de cinco intersecciones sin enumerar cada calle de manera individual. El bucle se desenrolla durante la compilación, con lo cual el programa resultante es equivalente a la declaración explícita de los cuatro tramos.

```
// Sin posición declarada: el generador las ubica por sí mismo.
intersection CruceRivadavia[5];

for i in 0 .. 3 {
    road Rivadavia[i] from CruceRivadavia[i] to CruceRivadavia[i + 1]
                        length 12;
}
```

Código 6.2: Generación paramétrica de un corredor mediante iteración acotada.

En el **Cód. (6.3)**, se define una *policy* que implementa un semáforo adaptativo: cuando la cola sobre la avenida principal supera un umbral configurable, la fase verde correspondiente se extiende y el evento queda registrado en la traza. Al ser parametrizada, la misma *policy* puede aplicarse sobre distintos semáforos de la red con umbrales diferentes.

```
// Los parámetros llevan tipo explícito, igual que las variables.
policy Adaptativa(road principal, integer umbral) {

    // El cuerpo se evalúa una vez por tick:
    if (queue(principal) > umbral) {
        extend principal by 5; // alarga la fase verde vigente
        log "Extendida {1} en el tick {2}", principal, tick;
    } else {
        keep; // deja el ciclo sin alterar
    }
}

light at CordobaCallao {
    phase green for CordobaOeste during 10;
    phase green for CordobaEste during 8;

    // Una misma policy admite umbrales distintos por semáforo:
    apply Adaptativa(CordobaOeste, 15);
}
```

Código 6.3: Semáforo adaptativo mediante una *policy* parametrizada.

Finalmente, en el **Cód. (6.4)**, se declara el bloque de reporte que selecciona las métricas a emitir en el artefacto JSON de salida, una vez finalizada la simulación.

```
report {
    completed; // vehículos que completaron su ruta
    average_travel_time; // tiempo promedio de viaje, en ticks
    peak_occupancy; // ocupación máxima por calle
}
```

Código 6.4: Selección de métricas para el reporte de salida.

Por último, en la **Fig. (6.1)** se muestra el artefacto HTML correspondiente al corredor del **Cód. (6.1)**, ampliado con una calle transversal sobre Callao, y obtenido mediante un prototipo del runtime construido para validar el diseño. Cada calle se representa como su secuencia de celdas, distinguiendo las libres de las ocupadas por un vehículo en movimiento o detenido; los semáforos exponen la fase vigente y su duración restante sobre la intersección que regulan; el panel lateral concentra las tres métricas junto con la ocupación por calle, y el registro inferior recoge tanto los eventos de la simulación como la traza emitida por las sentencias `log`.

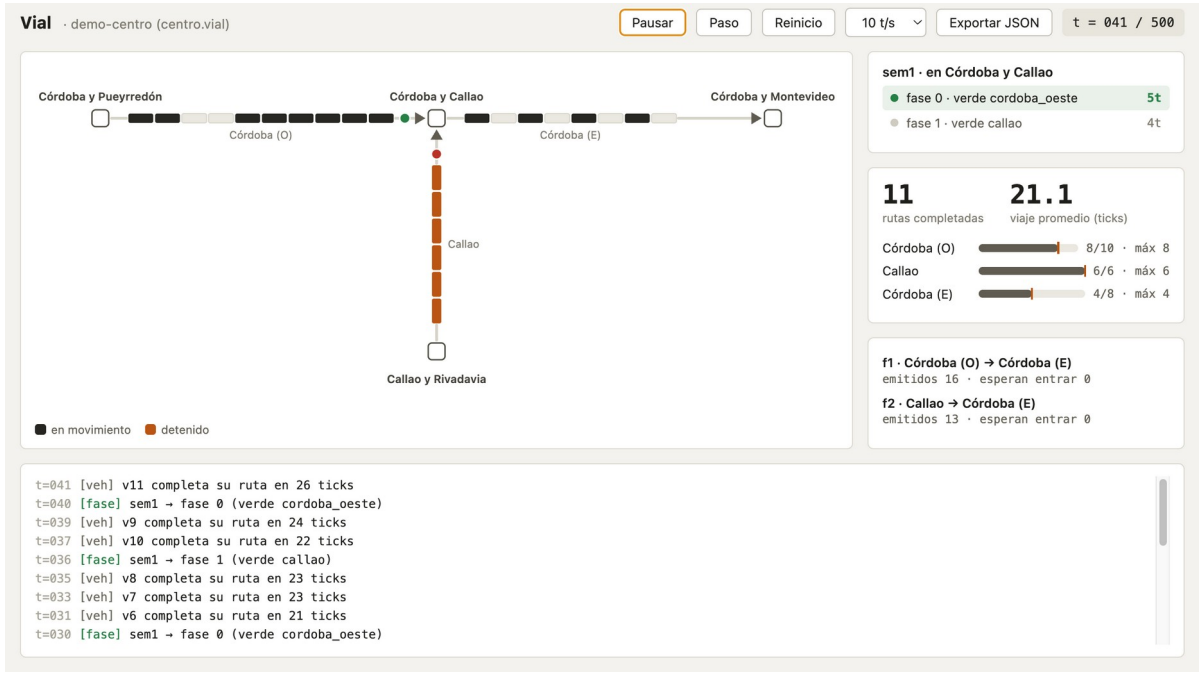


Figura 6.1: Artefacto HTML generado, ejecutándose sobre el prototipo del runtime.